



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

Informatik 1

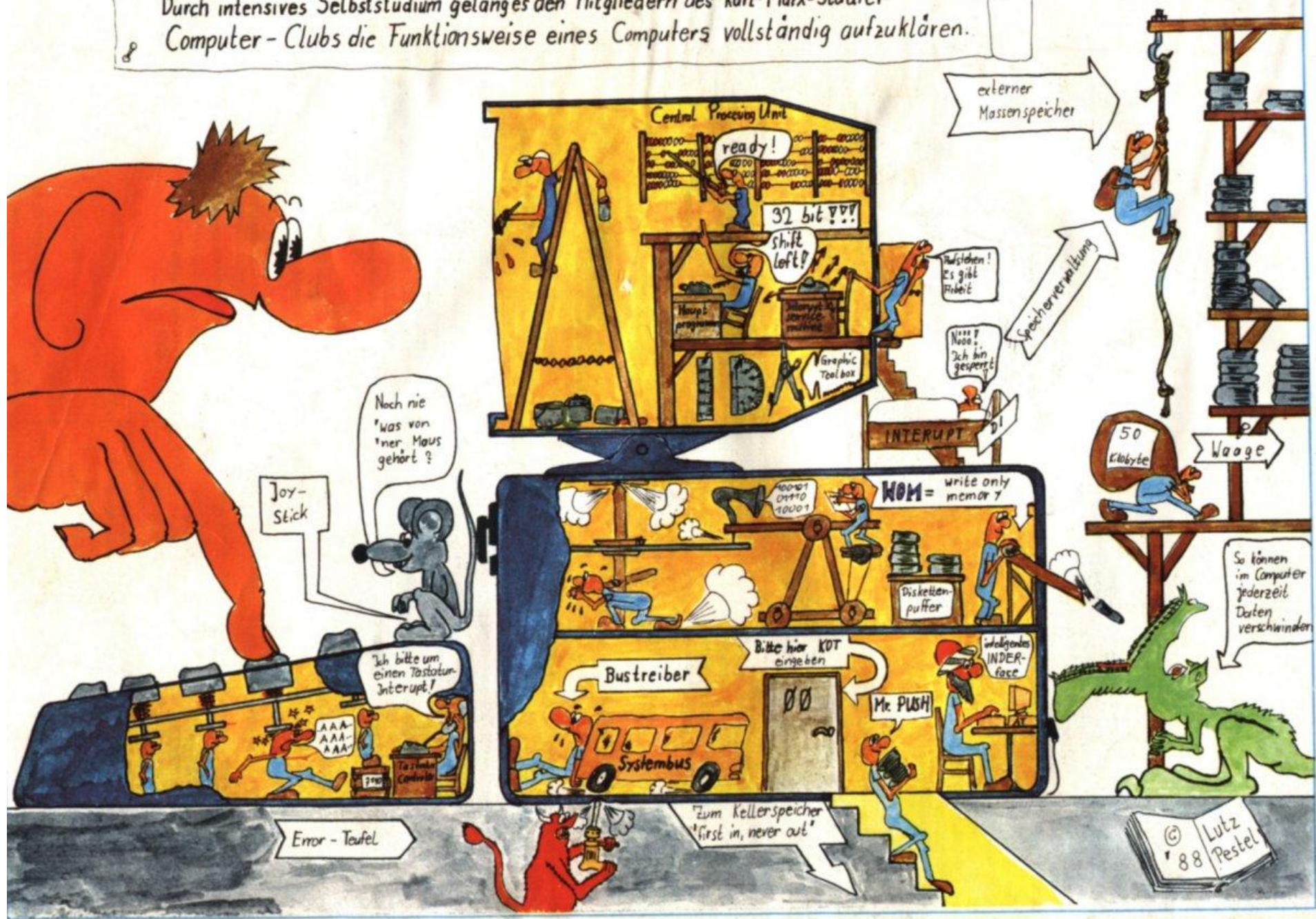
BA Medieninformatik

Prof. Dr.-Ing. Sabine Schumann

Fakultät Informatik und Digitale Gesellschaft, Fachgruppe Medieninformatik

Computerkomponenten & Betriebssysteme

Durch intensives Selbststudium gelangen den Mitgliedern des Karl-Marx-Städter Computer-Clubs die Funktionsweise eines Computers vollständig aufzuklären.



Was erwartet Sie in diesem Kapitel?

Computerkomponenten

- von-Neumann-Architektur
- Assembler

Betriebssysteme

- Prozesse / Tasks
- Multitasking
- Multithreading
- Scheduling-Verfahren

Prozess-Synchronisation

- Semaphore
- Monitor
- Deadlock

Was erwartet Sie in diesem Kapitel?

Computerkomponenten

- von-Neumann-Architektur
- Assembler

Betriebssysteme

- Prozesse / Tasks
- Multitasking
- Multithreading
- Scheduling-Verfahren

Prozess-Synchronisation

- Semaphore
- Monitor
- Deadlock

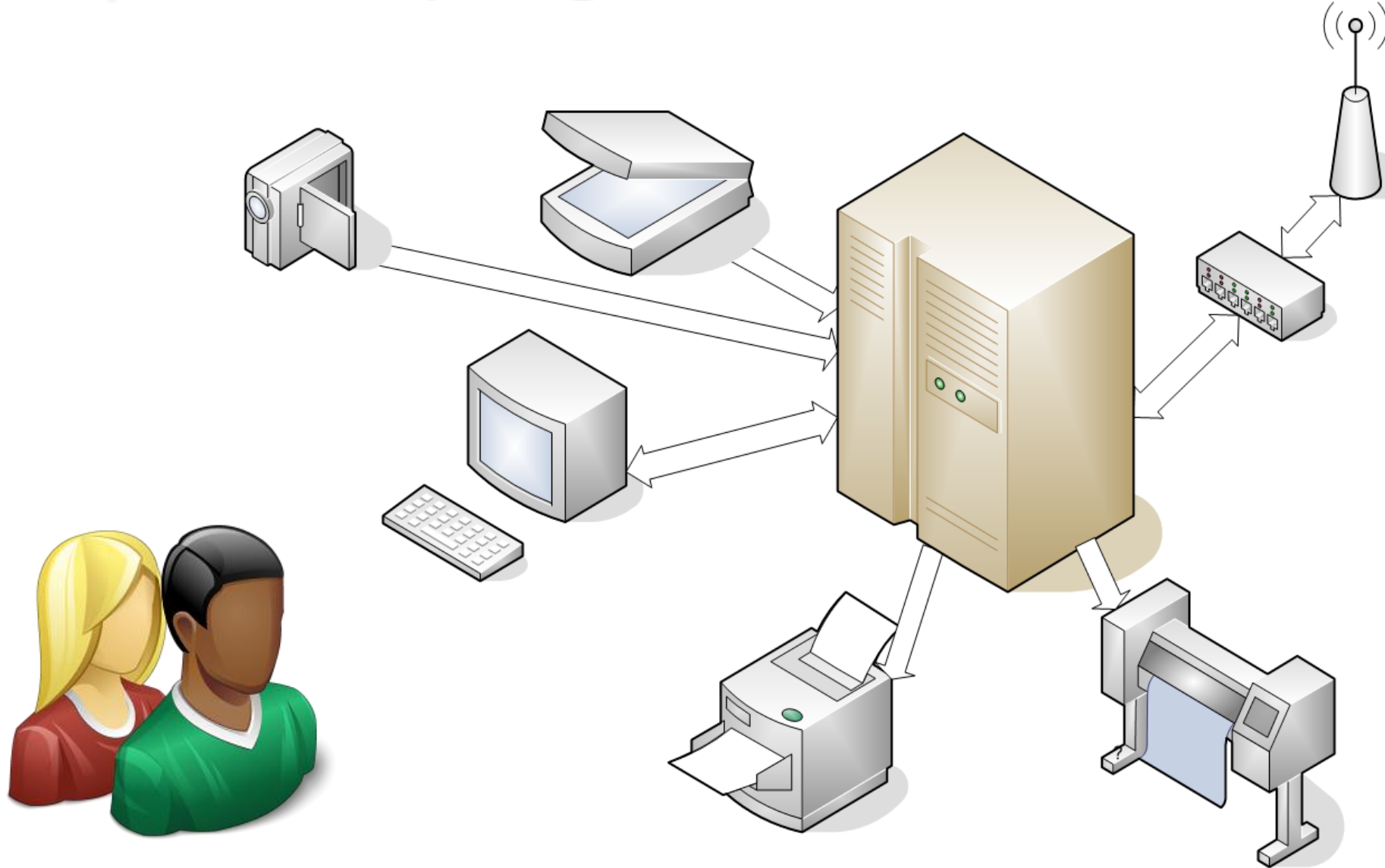


John von Neumann
(1903-1957)

Bildquelle: <https://www.3quarksdaily.com/3quarksdaily/2019/11/the-unparalleled-genius-of-john-von-neumann.html>

Computerkomponenten

Computer und Peripheriegeräte



Die Hardware ...

besteht grundsätzlich aus der Zentraleinheit (der Rechner) und Peripheriegeräten

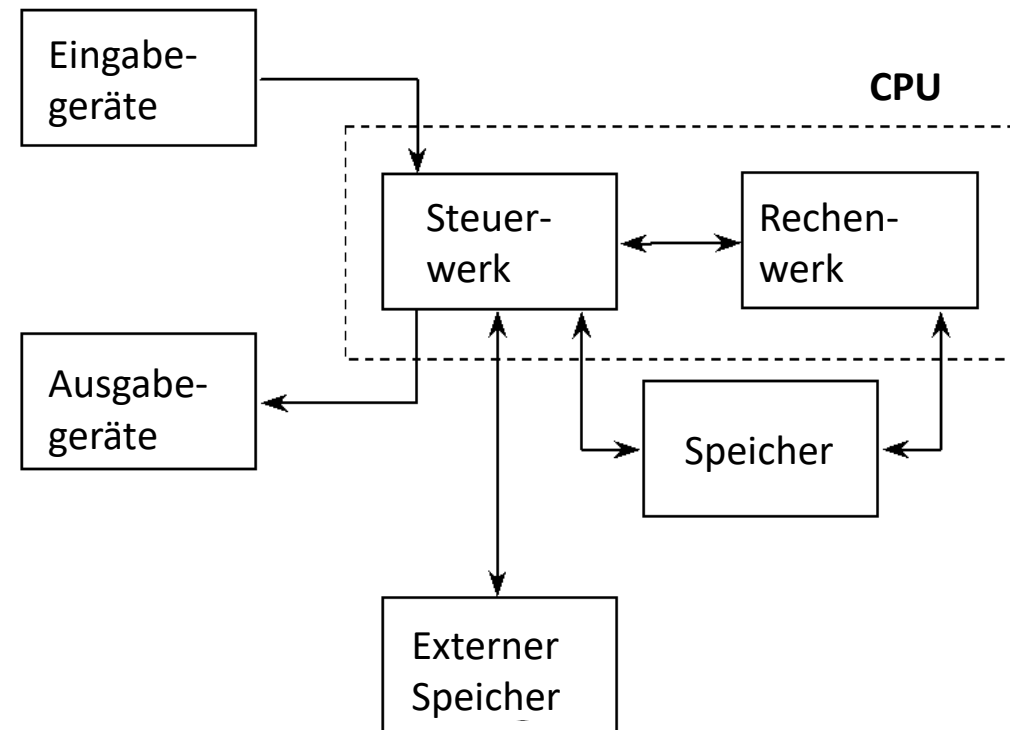
Peripheriegeräte werden unterteilt in

- Eingabegeräte
(Maus, Tastatur, Pen, Mikrofon, Datenhandschuh, ...)
- Ausgabegeräte
(Bildschirm, Beamer, Drucker, Lautsprecher, ...)
- Ein- und Ausgabegeräte
(Festplatte, Netzwerkkarte, Modem, DVD-Laufwerk...)

von-Neumann-Architektur

siehe auch [Folien](#) zur Historie

„First Draft of a Report on the EDVAC“ 30. Juni 1945



John von Neumann

Bildquelle: By LANL [Public domain or Public domain], via Wikimedia Commons <https://commons.wikimedia.org/wiki/File:JohnvonNeumann-LosAlamos.gif>

Aufgabe

Führen Sie eine Recherche durch!

- Was ist der von-Neumann-Flaschenhals (engl. bottleneck)?
- Welche Auswirkungen hat er?



15 Minuten

EVA – Prinzip

Eingabe, **V**erarbeitung, **A**usgabe

Computer-generation	Eingabeeinheit	Verarbeitungseinheit	Ausgabeeinheit
Röhrenrechner	Schalttafeln	Trommelspeicher	Leuchtanzeigen, Drucker mit Endlospapier
Transistorrechner	Lochkarten	Magnetkernspeicher	Drucker mit Endlospapier
IC-Rechner	Terminal mit Tastatur, Magnetband	IC-Schaltungen	Terminal, Drucker mit Endlospapier, Magnetband
Mikrocomputer	Tastatur, Maus, Festplatte, etc.	Mikroprozessor	Bildschirm, Festplatte, Laserdrucker, ...

Zentraleinheit (CPU)

Hauptplatine (Mainboard, Motherboard) enthält

- Mikroprozessor (CPU)
- RAM (Random Access Memory)
- ROM (Read-only Memory)
- Busse und Schnittstellen
- Chipsatz für die Steuerung sämtlicher Anschlüsse

heutzutage auch durchaus Grafik-, Netzwerk-, Soundkarte ...

... diese sind jedoch trotzdem Peripheriegeräte und nicht (nur wegen ihrer Unterbringung)
Bestandteil des Steuerwerks

Zentraleinheit - Mikroprozessor

- integrierte Schaltkreise
- ursprünglich ein paar Tausend Transistoren
- heute ... dank zunehmender Integrationsdichte ...

Grundaufbau:

Steuerwerk (Befehlskodierung und Ablaufsteuerung)

- Befehlszählerregister
- Befehlsregister (Ausführung)
- Zustandsregister

Rechenwerk (Addierwerk mit Verknüpfungslogik) (ALU)

- Arbeitsregister 1... n
- Statusregister

Der Prozessor ist durch Bussysteme mit den anderen Komponenten des Mainboard verbunden.

Bussysteme

Bussysteme sind

- Datenbus
Austausch von Daten mit dem Arbeitsspeicher
- Adressbus
übertragen von Speicheradressen
- Steuerbus
Ansteuerung der Anschlüsse für die Peripheriegeräte



Register (1)

- prozessorientierte Speicherplätze innerhalb des Prozessorkerns
- können jeweils ein binäres Datum einer bestimmten Länge (z.B. 32-Bit) aufnehmen
- Register sind mit anderen Prozessor-Komponenten verbunden (Ablaufsteuerung, Verknüpfungslogik)

man unterteilt Register in

- Arbeitsregister
- Befehlszählerregister
- Befehlsregister

Register (2)

Arbeitsregister

- nur mit Daten in Arbeitsregistern können direkte (logische) Operationen durchgeführt werden (z.B. 2 Zahlen addieren; 2 Zahlen miteinander vergleichen; ...)
- haben einen „Namen“, über den Sie angesprochen werden (D1...D7 oder EAX...EDX)
- können Daten aufnehmen (*Datenregister*)
- können Adressen aufnehmen (u.U. nicht alle, abhängig vom Prozessortyp) (*Adressregister*)

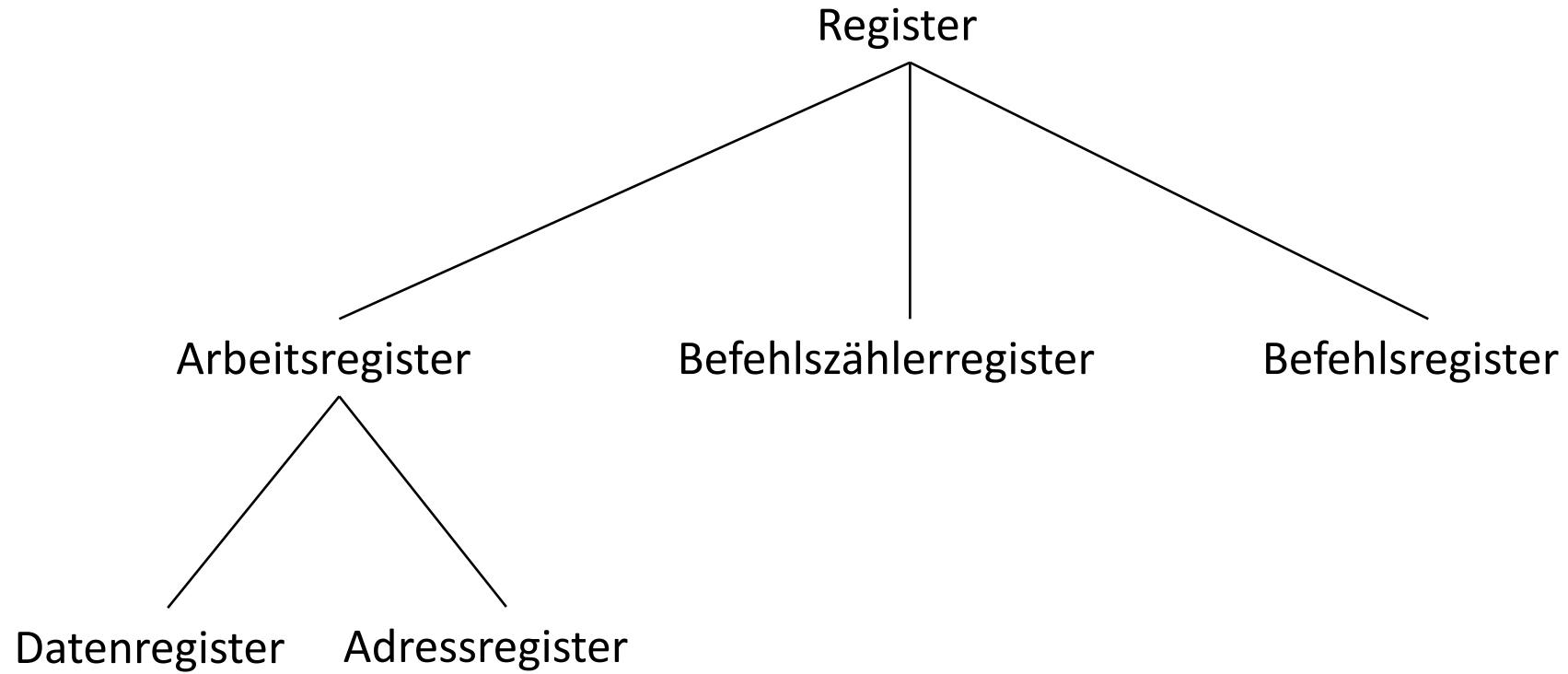
Befehlszählerregister

- beinhaltet immer die Adresse des nächsten auszuführenden Befehls (*Instruction Pointer, IP*)

Befehlsregister

- nehmen einen binären Maschinenbefehl auf

Überblick Register



Register - Leistungsfähigkeit von Rechnern (1)

Einfluss von Registern auf die Leistungsfähigkeit von Rechnern:

- **Größe der Arbeitsregister** legen die größte mit einem Befehl verarbeitbare ganze Zahl fest.

8-Bit-Prozessor: $2^8 = 256$

32-Bit-Prozessor: $2^{32} = 4\,294\,967\,296$

größere Zahlen müssen in mehreren Befehlen unter Berücksichtigung des Übertrages berechnet werden, was die Bearbeitungsgeschwindigkeit verringert

- **Größe der Befehlszählerregister** und **Adressregister** legen die maximale Größe des adressierbaren Arbeitsspeicher fest

16-Bit: $2^{16} = 64$ KByte,

32-Bit: $2^{32} = 4$ GByte ansprechbarer Arbeitsspeicher

Register - Leistungsfähigkeit von Rechnern (2)

Größe der Befehlsregister wirkt sich auf Anzahl und Komplexität der möglichen Maschinenbefehle aus

je kleiner Registerlängen sind...

... umso kostengünstiger kann der Prozessor hergestellt werden,

... andererseits ist die Leistungsfähigkeit natürlich geringer

Assembler

- binäre Maschinenbefehle in den Befehlsregistern sind für den Menschen praktisch unlesbar



Einführung symbolischer Maschinenbefehle

- jedem binären Maschinenbefehl wird ein Symbol mit entsprechenden Operandenzusätzen zugeordnet



Assembler

- Maschinenbefehle bestehen i.a. aus
 - der eigentliche Befehl (OP-Code / Operationscode)
 - Operandenteil mit Angabe der Adressierungsart
 - Operandenwert oder eine Adresse
- je nach Art des Befehls können Maschinenbefehle auch verschiedene Längen und unterschiedliche Anzahl von Operanden haben

Assembler – Beispiele

Beispiele:

- `mov BX, $7A35`
hole aus dem Arbeitsspeicher den Wert, der an der Adresse 7A35 steht und lege ihn im Arbeitsregister BX ab
- `add BX, 20`
addiere den Wert 20 zum Inhalt des Arbeitsregisters BX
- `cmp BX, 50`
vergleiche den Wert im Register BX mit 50
falls im Register BX 50 steht, wird ein bestimmtes Bit (Flag) im Zustandsregister gesetzt
- `je $B7F4` (jump if equals)
falls der vorherige Vergleich *gleich* ergeben hat (entsprechendes Flag im Zustandsregister), springe zur Programmadresse B7F4

Unterteilung Maschinenbefehle

- Arithmetische Befehle (add, inc, ...)
- Logische Befehle (and, ...)
- Sprungbefehle (jmp, call, ...)
- Transportbefehle (mov, ...)
- Prozessorkontrollbefehle (cli)
dienen der Priorisierung der Aufgaben und zur internen Organisation im Prozessor

PDP 11

PDP-11-Baureihe wurde 1970 begonnen



Bildquelle: PDP 11 im Technischen Museum Wien, By Stefan_Kögl [GFDL (<http://www.gnu.org/copyleft/fdl.html>)]

Assembler (PDP 11) (1)

Address	Int Vec- tor	Prior- ity	NPR
776 700	254*	BR5**	X
776 702			
776 704			
776 706			
776 710			
776 712			
776 714			
776 716			
776 720			
776 722			
776 724			
776 726			
776 730			
776 732			
776 734			
776 736			
776 740			
776 744			
776 746			
776 750+			
776 752+			
204*	BR5**	X	
772 040			
772 042			
772 044			
772 046			
772 050			
772 052			
772 054			
772 056			
772 060			
772 062			
772 064			
772 066			
772 070+			
772 072+			
224*	BR5**	X	
772 440			
772 442			
772 444			
772 446			
772 450			
772 452			
772 454			
772 456			
772 460			
772 462			
772 464			
772 466			
772 470			
772 472			
772 474+			
772 476+			
772 520			
772 522			
772 524			
772 526			
772 530			
772 532			

Device	Registers	Address	Int Vec- tor	Prior- ity	NPR
TA11	Cassette command & status data buffer	(TACS) (TADB)	777 500 777 502	260	BR6
TC11/ TU56	DECtape control & status command word count bus address data	(TCST) (TCCM) (TCWC) (TCBA) (TCDT)	777 340 777 342 777 344 777 346 777 350	214	BR6 X

BM873-YA BOOTSTRAP LOADER:

Starting Address	Device
773 000	RF11
773 010	RK11
773 020	Transfer to address contained in Switch Register
773 030	TC11
773 050	TM11
773 100	RP11
773 144	RC11
773 210	ASR paper tape reader
773 230	TA11
773 312	PC11

BM873-YB BOOTSTRAP LOADER:

Starting Address	Device
773 000	RJS03/RJS04
773 002	RJS03/RJS04
773 030	RK11
773 032	RK11
773 070	TC11
773 110	TM11
773 136	RF11
773 150	TJU16
773 212	RC11
773 320	RJP04
773 322	RJP04
773 344	Transfer to address in console switch register
773 350	RP11
773 352	RP11
773 510	KL11/DL11
773 524	TA11
773 526	TA11
773 620	PC11

PDP-11/70 BOOTSTRAP LOADER:

Starting Address	17	765	000
Device Type:	1	TM11	6
	2	TC11	7
	3	RK11	10
	4	RP11	11

ABSOLUTE LOADER

Starting Address: — 500
Memory Size: —

4K	017
8K	037
12K	057
16K	077
20K	117
24K	137
28K	157
(or larger)	

BOOTSTRAP LOADER

Address	Contents	Address	Contents
— 744	016 701	— 764	000 002
— 746	000 026	— 766	— 400
— 750	012 702	— 770	005 267
— 752	000 352	— 772	177 756
— 754	005 211	— 774	000 765
— 756	105 711	— 776	177 560 (TTY) or 177 550 (PC11)
— 760	100 376		
— 762	116 152		

773 000 Paper Tape Bootstrap
773 100 Disk/DECtape Bootstrap
773 200 Card Reader Bootstrap
773 300 Cassette Bootstrap
773 400 Floppy Disk Bootstrap

MR11-DB BOOTSTRAP LOADER:

Starting Address	Device
773 100	RF11
773 110	RK11
773 120	TC11
773 136	TM11
773 154	RP11
773 220	RC11

7-BIT ASCII CODE:

Octal Code	Char	Octal Code	Char	Octal Code	Char	Octal Code	Char
000	NUL	040	SP	100	@	140	~
001	SOH	041	!	101	A	141	a
002	STX	042	"	102	B	142	b
003	ETX	043	#	103	C	143	c
004	EOT	044	\$	104	D	144	d
005	ENQ	045	%	105	E	145	e
006	ACK	046	&	106	F	146	f
007	BEL	047	'	107	G	147	g
010	BS	050	(	110	H	150	h
011	HT	051	)	111	I	151	i
012	LI	052	*	112	J	152	j
013	VT	053	+	113	K	153	k
014	FF	054	,	114	L	154	l
015	CR	055	-	115	M	155	m
016	SO	056	.	116	N	156	n
017	SI	057	/	117	O	157	o
020	DLE	060	0	120	P	160	p
021	DC1	061	1	121	Q	161	q
022	DC2	062	2	122	R	162	r
023	DC3	063	3	123	S	163	s
024	DC4	064	4	124	T	164	t
025	NAK	065	5	125	U	165	u
026	SYN	066	6	126	V	166	v
027	ETB	067	7	127	W	167	w
030	CAN	070	8	130	X	170	x
031	EM	071	9	131	Y	171	y
032	SUB	072	:	132	Z	172	z
033	ESC	073	;	133	[	173	{
034	FS	074	<	134	\	174	
035	GS	075	=	135	]	175	}
036	RS	076	>	136	^	176	~
037	US	077	?	137	_	177	DEL

digital pdp11

PROGRAMMING CARD

FOR FAMILY OF PDP-11 COMPUTERS

WORD FORMAT:

1 16 12 11 9 8 6 5 3 2 0

BINARY OCTAL
REPRESENTATION

MODE

Mode	Name	Symbolic	Description
0	register	R	(R) is operand [ex. R2—%2]
1	register deferred	(R)	(R) is address
2	auto-increment	(R)++	(R) is adrs; (R) + (1 or 2)
3	auto-incr deferred	@(R)++	(R) is adrs of adrs; (R) + 2
4	auto-decrement	--(R)	(R) — (1 or 2); (R) is adrs
5	auto-decr deferred	@--(R)	(R) — 2; (R) is adrs of adrs
6	index	X(R)	(R) + X is adrs
7	index deferred	@X(R)	(R) + X is adrs of adrs

PROGRAM COUNTER ADDRESSING: Reg = 7

MODE

2	immediate	≡n	operand n follows instr
3	absolute	@≡A	address A follows instr
6	relative	A	instr adrs + 4 + X is adrs
7	relative deferred	@A	instr adrs + 4 + X is adrs of adrs

LEGEND:

Op Codes	Operations
n = 0 for word/1 for byte	() = contents of
SS = source field (6 bits)	s = contents of source
DD = destination field (6 bits)	d = contents of destination
R = register (3 bits; 0 to 7)	r = contents of register
XXX = offset (8 bits); +127 to -128	+ = becomes
N = number (3 bits)	X = relative address
NN = number (6 bits)	% = register definition

Boolean

A = AND
V = inclusive OR
Y = exclusive OR
~ = NOT

Condition Codes

* = conditionally set/cleared
— = not affected
0 = cleared
1 = set

NOTE:

▲ — Applies to the 11/35, 11/40, 11/45 & 11/70 computers
● — Applies to the 11/45 & 11/70 computers

digital equipment corporation

MAYNARD, MASSACHUSETTS

July 1975

Assembler (PDP 11) (2)

R0: 59eb R1: d5f7 R2: 6cd7 R3: 806b R4: 0000 R5: 0000 SP: 0000 PC: 0052

Code

```
add KEY_2L r3
adc r2
add KEY_2H r2

; Z2 = (y + sum)

mov Y_HIGH r0
mov Y_LOW r1

add SUM_L r1
adc r0
add SUM_H r0

; Z4 = Z1 xor Z2

xor r0 r2
xor r1 r3

; Z3 = ((y>>5) + key[3])

mov Y_HIGH r0
mov Y_LOW r1

; right shift = sign bit is replicated

ashc #-5 r0

; use BIC to clear unwanted replication

bic #174000 r0

add KEY_3L r1
```

Output

SUM_L	0204	0000
DEL_H	0206	9e37
DEL_L	0208	79b9
KEY_0H	020a	1122
KEY_0L	020c	3344
KEY_1H	020e	5566
KEY_1L	0210	7788
KEY_2H	0212	9900
KEY_2L	0214	aabb
KEY_3H	0216	ccdd
KEY_3L	0218	eeff
N	021a	0020
V_0H	021c	1234
V_0L	021e	5678
V_1H	0220	90ab
V_1L	0222	cdef
CTX_0H	0224	8df5
CTX_0L	0226	d71f
CTX_1H	0228	9c3b
CTX_1L	022a	e43f
PTX_0H	022c	1234
PTX_0L	022e	5678
PTX_1H	0230	90ab
PTX_1L	0232	cdef

Condition Codes:
N = 1
Z = 0
V = 0
C = 0

Compile Run Step Watch View Binary Clear

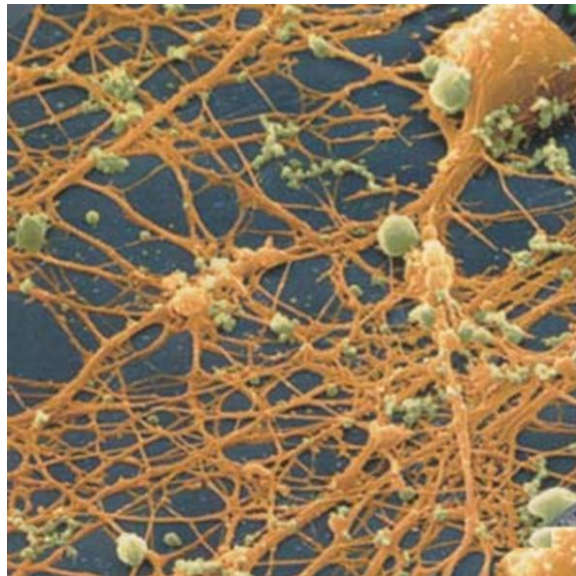
Reset Machine Hex|Oct Jump 0 << >>

Alternative Rechnerarchitekturen

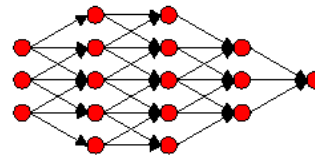
Datenverarbeitung mittels künstlichen neuronalen Netzen

KNN:

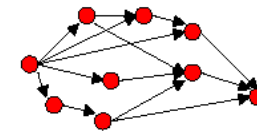
- vereinfachte Nachbildung des zentralen Nervensystems von Lebewesen
- Netzwerk aus (vielfach) miteinander verbundenen Neuronen
- müssen lernen
(zu Eingabedaten gewünschtes Ausgabeverhalten)



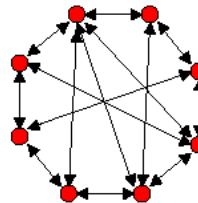
• **Multilayer Model**



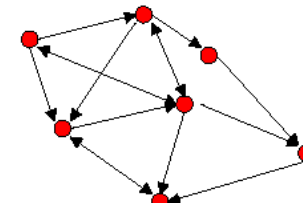
• **Feed-Forward Model**



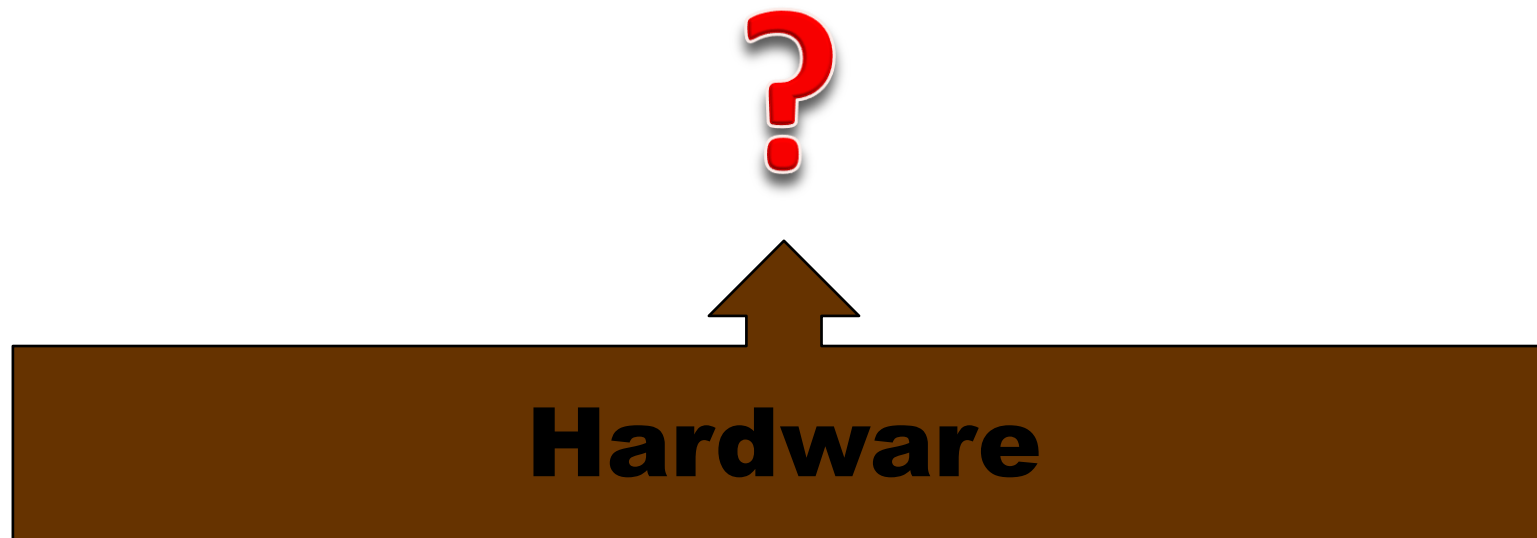
• **Recurrent (Relaxation) Model, symmetric**



• **Recurrent (Relaxation) Model, asymmetric**



[Malakka]



Betriebssysteme

Was erwartet Sie in diesem Kapitel?

Computerkomponenten

- von-Neumann-Architektur
- Assembler

Betriebssysteme

- Prozesse / Tasks
- Multitasking
- Multithreading
- Scheduling-Verfahren

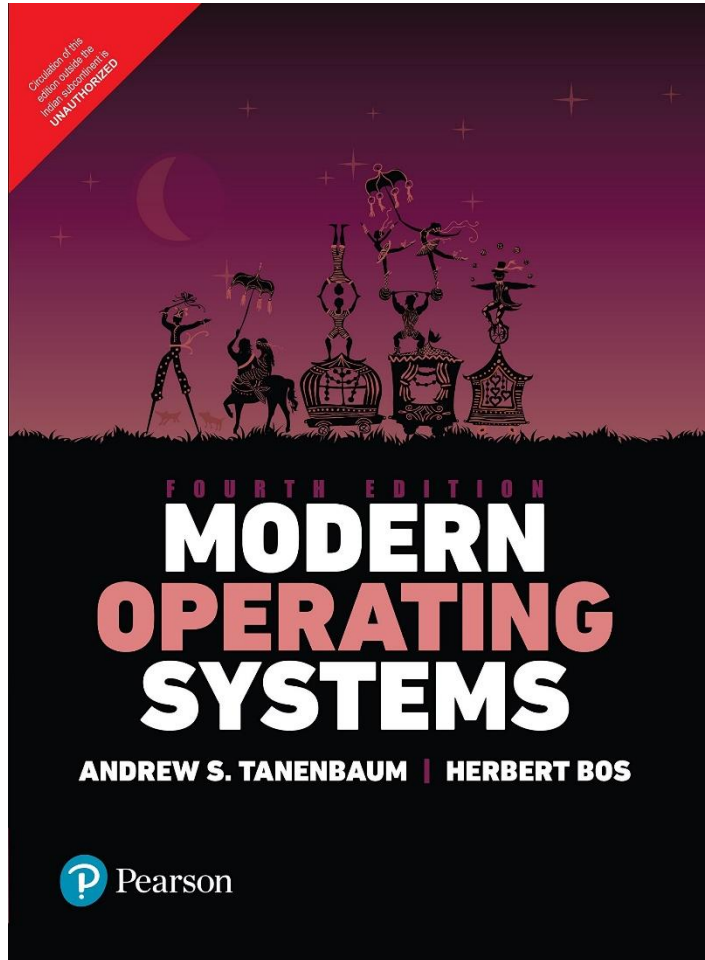
Prozess-Synchronisation

- Semaphore
- Monitor
- Deadlock





Literatur



Andrew S. Tanenbaum, Herbert Bos:
Modern Operating Systems.
4. Aufl., Pearson, 2016

Definition Betriebssystem

Ein **Betriebssystem**

(auch OS vom englischen *operating system*)

- ist eine Zusammenstellung von Computerprogrammen, die die Systemressourcen eines Computers wie Arbeitsspeicher, Festplatten, Ein- und Ausgabegeräte verwaltet und diese Anwendungsprogrammen zur Verfügung stellt.
- Das Betriebssystem bildet dadurch die Schnittstelle zwischen den Hardware-Komponenten und der Anwendungssoftware des Benutzers.

(nach Wikipedia)

Aufgaben eines Betriebssystems

- Prozessverwaltung
(Laden, Ausführen, Unterbrechen, Beenden von Programmen, Verwaltung und Zuteilung von Prozessorzeit, ...)
- Speicherverwaltung
- Dateiverwaltung
- Netzwerk
- Sicherheit (Speicherschutz, Nutzerrechte)
- Geräteverwaltung (Treiber, ...)
- (graphische Nutzeroberflächen)

nochmal zur PDP 11

Dennis Ritchie (1941-2011) und Ken Thompson (*1943)
arbeiten an der PDP 11

(Bell Telephone Laboratories, 1972)

The Thompson and Ritchie Story:

<https://www.youtube.com/watch?v=g3jOJfrOknA>

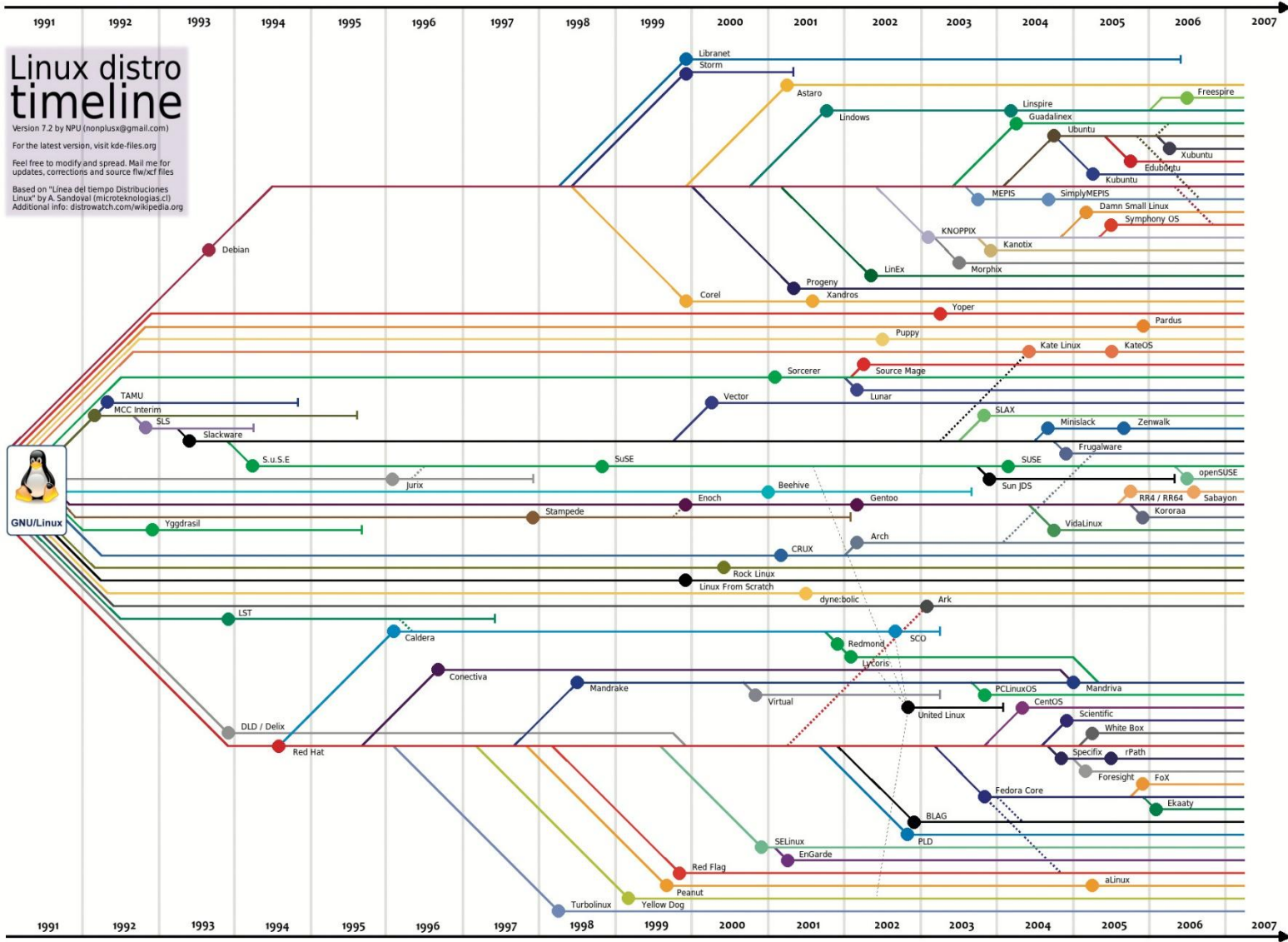


Bildquelle: <http://www.computerhistory.org/fellowawards/hall/ken-thompson/>

STEVE JOBS VS. DENNIS RITCHIE



Linux Timeline

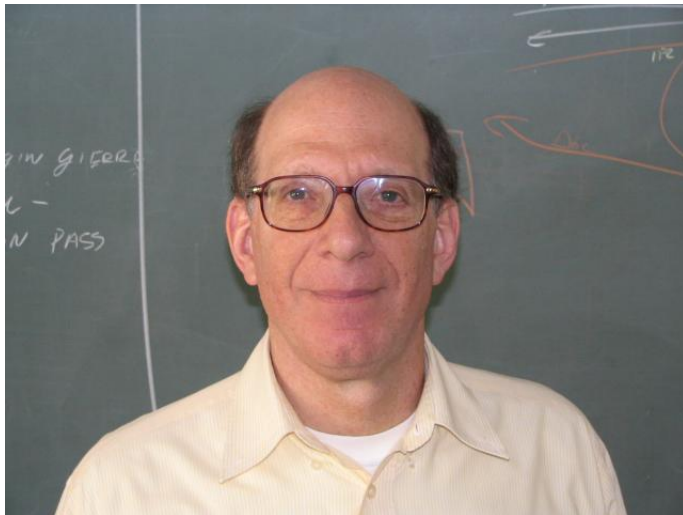


Bildquelle: https://www.heise.de/select/ct/2019/27/1572613284730930/contentimages/linux-distro-timeline_dw_SC.jpg

Prozesse / Tasks

- Anwendungsprogramme werden in Form von Prozessen verwaltet und zum Ablauf gebracht
- das Programm beinhaltet die Anweisungen
- der Prozess wird gestartet und führt das Programm aus
- bedeutet:
der Prozess kann auch zwischenzeitlich unterbrochen werden
(um andere Prozesse laufen zu lassen)

Andrew Stuart Tanenbaum (*1944)



“Consider a culinary-minded computer scientist who is baking a birthday cake for his daughter.

He has a birthday cake recipe and a kitchen well stocked with all the input: flour, eggs, sugar, extract of vanilla, and so on.

In this analogy, the recipe is the program (i.e., an algorithm expressed in some suitable notation), the computer scientist is the processor (CPU), and the cake ingredients are the input data.

The process is the activity consisting of our baker reading the recipe, fetching the ingredients, and baking the cake.”

Multitasking / Scheduling

... ist ein Verfahren bei dem sich mehrere Aufgaben, auch **Prozesse** genannt, gemeinsame Verarbeitungsressourcen wie eine CPU teilen

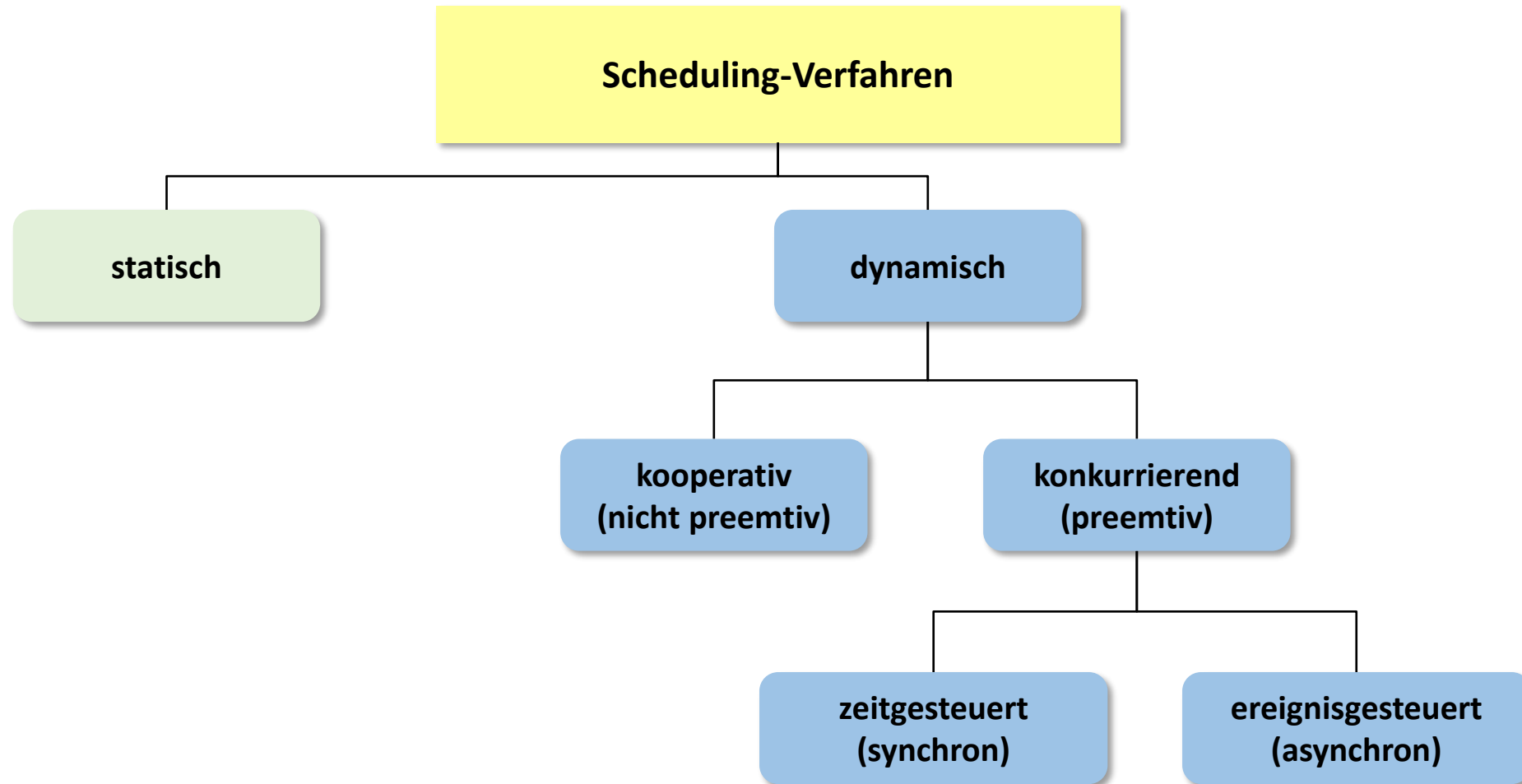
- mehrere Prozesse können abwechselnd bearbeitet werden
- Auswahl und Umschaltung des nächsten Prozesses erfolgt durch das Scheduler- bzw. Dispatcher-Programm des Betriebssystems
- der Prozess, der unterbrochen wird, muss „zwischengespeichert“ werden, damit er später wieder weiterlaufen kann

Bei einem Computer mit einer einzigen CPU läuft zu jedem Zeitpunkt nur eine Task, d.h. die CPU führt aktiv Anweisungen für diese Task aus.

Multithreading

- entstanden aus der Idee, dass der effizienteste Weg für kooperierende Prozesse zum Datenaustausch darin besteht, ihren gesamten Speicherplatz zu teilen
- d.h. innerhalb desselben Programmes parallel arbeiten
- somit sind Threads im Grunde genommen Prozesse, die im gleichen Speicherkontext laufen (quasi-parallel laufende Ablauffäden innerhalb eines Prozesses)
- das Wechseln zwischen Threads ist nicht mit einer Änderung des Speicherkontextes verbunden, damit entfällt aber auch der Speicherschutz

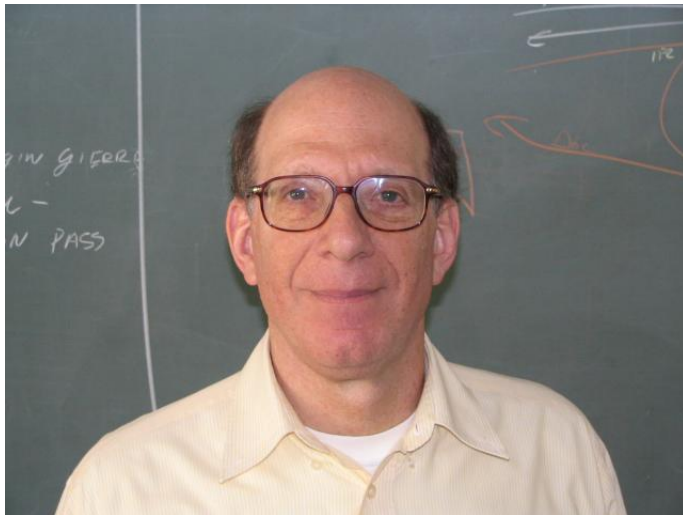
Scheduling-Verfahren (1)



Scheduling-Verfahren (2)

- kooperativ
Prozess muss den Prozessor *freiwillig* abgeben
- konkurrierend
 - Scheduler entzieht dem Prozess den Prozessor
 - damit ist der Prozess erstmal vorzeitig unterbrochen worden (Preemption)
 - Scheduler übernimmt die Sicherung des anzuhaltenden Prozesses
 - Scheduler muss den nun zur Weiterverarbeitung ausgewählten Prozess wiederherstellen (restaurieren)
 - dafür gibt es 2 Möglichkeiten:
 - zeitgesteuert (zu festen bzw. variablen Zeiten)
Prozesse erhalten i.d.R. Rechenzeit in abwechselnder Reihenfolge
 - ereignisgesteuert (bei Eintritt bestimmter Ereignisse)
Prozesse erhalten i.d.R. Rechenzeit nach einer (vorher festgelegten) Priorität

Andrew Stuart Tanenbaum (*1944)



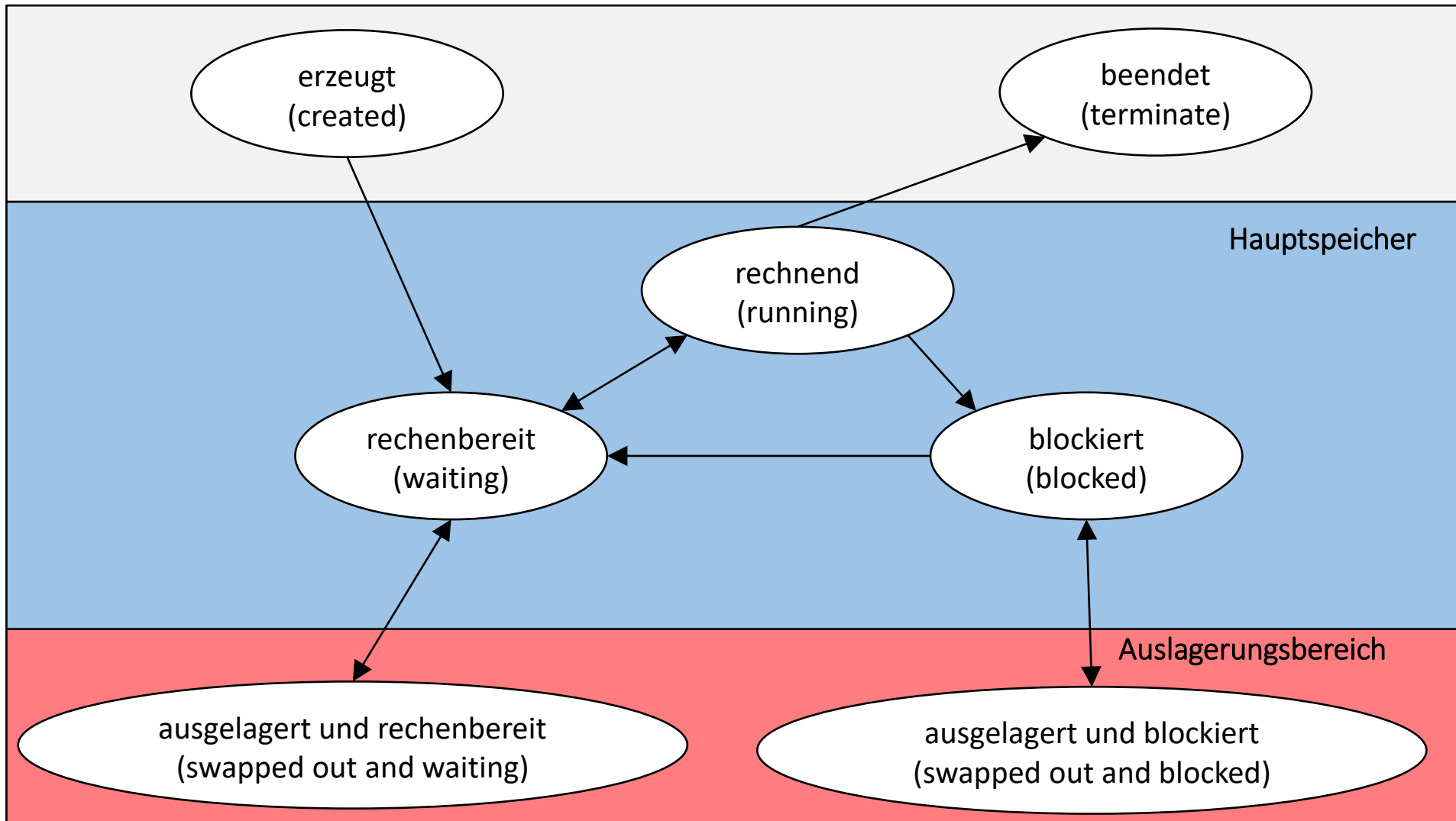
“Now imagine that the computer scientist's son comes running in screaming his head off, saying that he has been stung by a bee.

The computer scientist records where he was in the recipe (the state of the current process is saved), gets out a first aid book, and begins following the directions in it.

Here we see the processor being switched from one process (baking) to a higher-priority process (administering medical care), each having a different program (recipe versus first aid book).

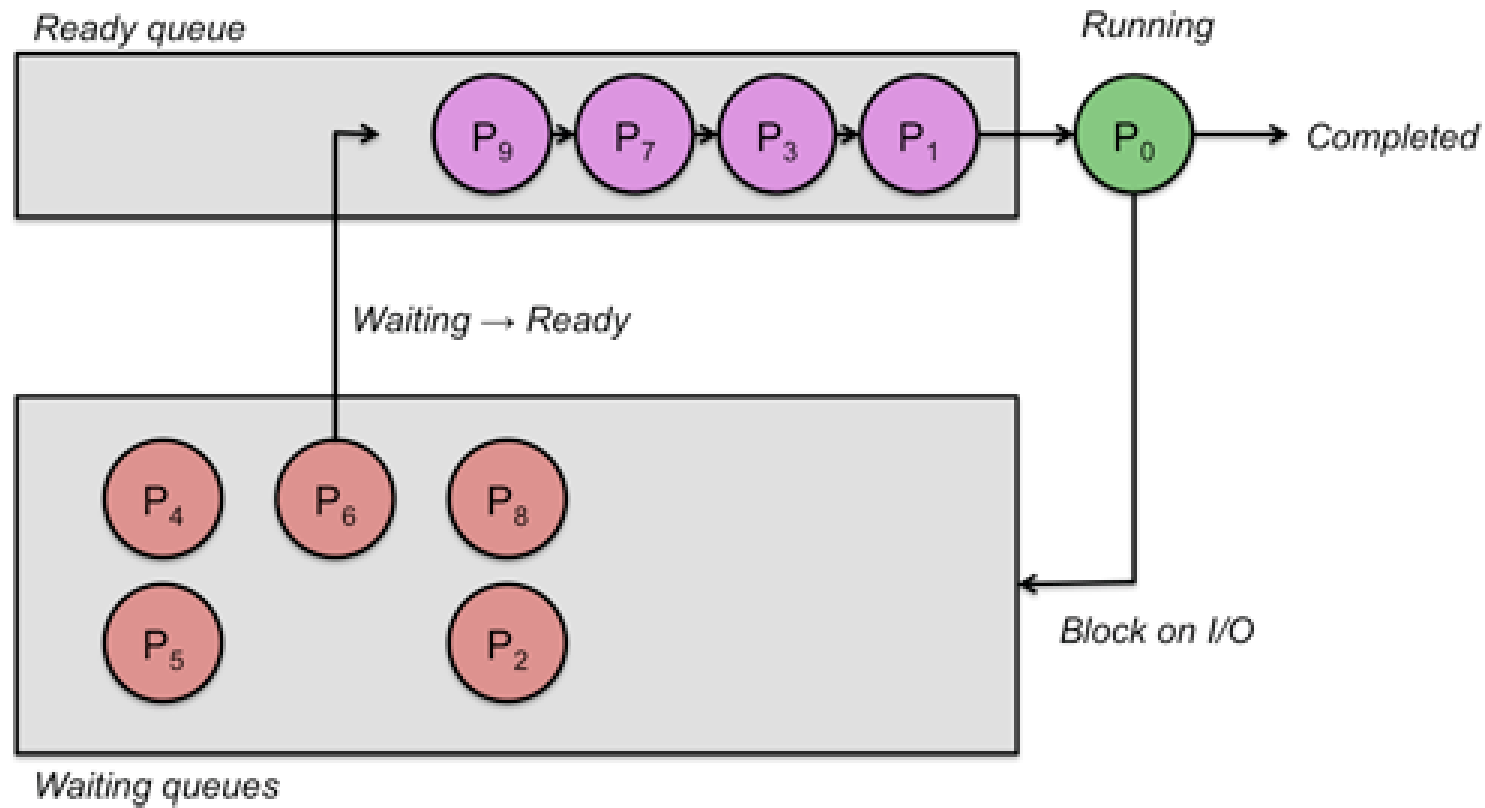
When the bee sting has been taken care of, the computer scientist goes back to his cake, continuing at the point where he left off. “

Prozesszustände im Zustandsdiagramm



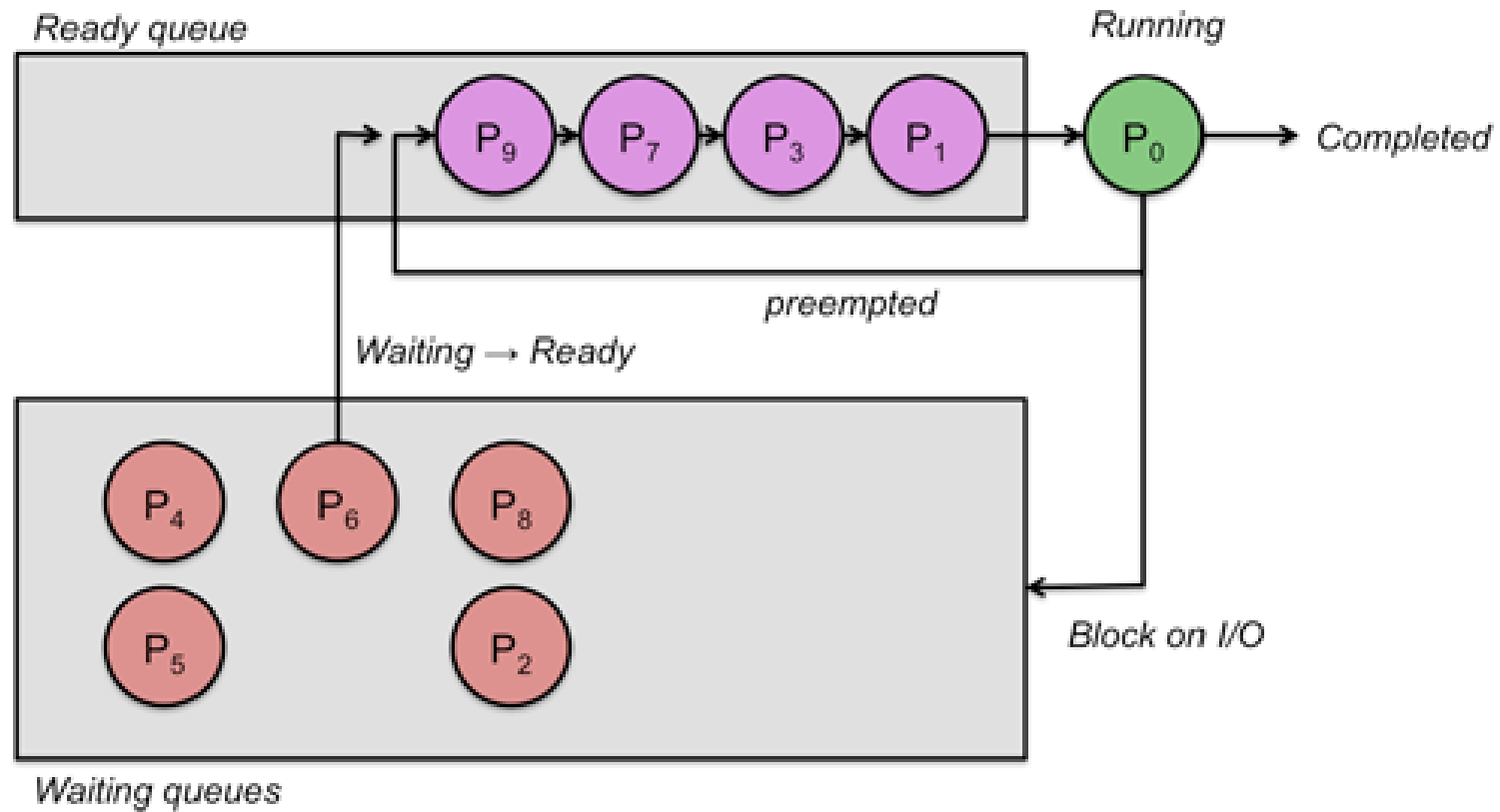
First-Come, First-Served Scheduling

- nicht preemtiv

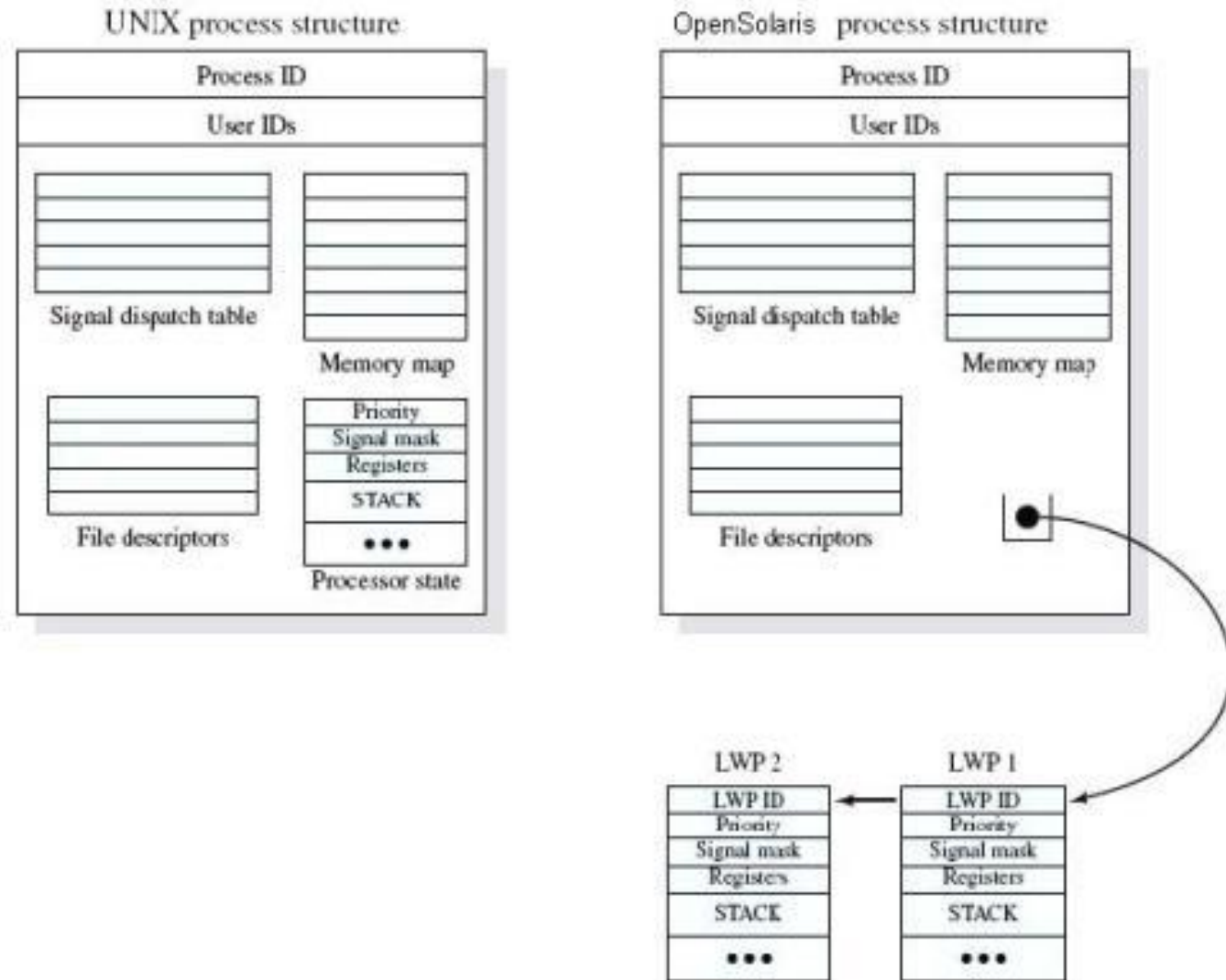


Round robin Scheduling

- preemptiv



Prozessbeschreibungsblock



Dingze Zhu, 2009, <https://res.infoq.com/articles/kernel-comparison-unix-zhu/en/resources/image3.jpg>

Kritische Abschnitte

```
int std=11, min=59
```

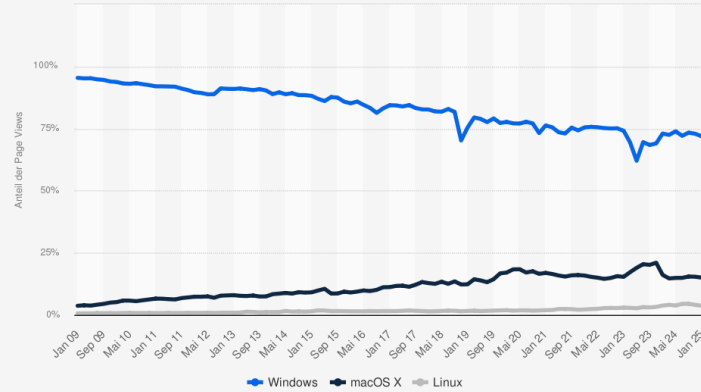
```
/* Task 1 */  
min=min+1;  
if (min==60) {  
    std=std+1;  
/* Hier wird Task 1 von Task 2 abgebrochen */  
    min=0;  
}
```



```
/* Task 2 */  
Ausgabe:  
Uhrzeit=std:min
```

Betriebssysteme ...

Marktanteile der führenden Betriebssysteme weltweit von Januar 2009 bis Januar 2025

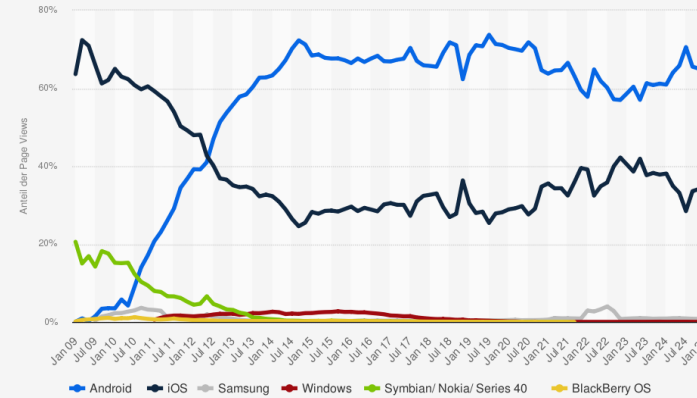


Quelle:
StatCounter
© Statista 2025

Weitere Informationen:
Weltweit; Desktop/Laptop

statista

Marktanteile der führenden mobilen Betriebssysteme an der Internetnutzung mit Mobiltelefonen in Deutschland von Januar 2009 bis Januar 2025

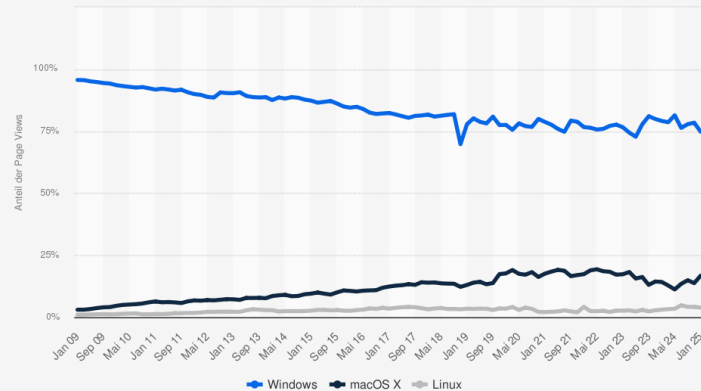


Quelle:
StatCounter
© Statista 2025

Weitere Informationen:
Deutschland

statista

Marktanteile der führenden Betriebssysteme in Deutschland von Januar 2009 bis Januar 2025

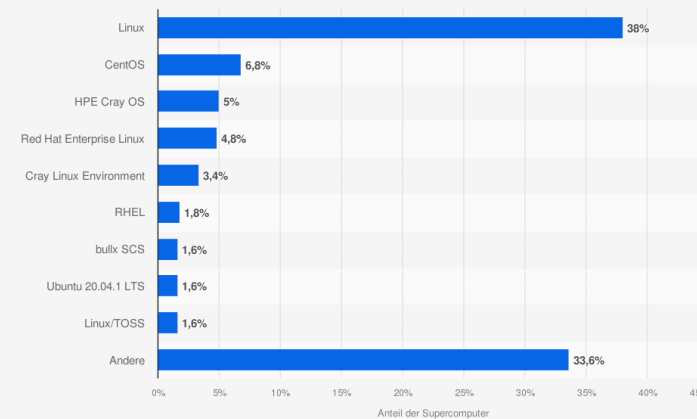


Quelle:
StatCounter
© Statista 2025

Weitere Informationen:
Deutschland; Desktop/Laptop

statista

Verteilung der 500 leistungsstärksten Supercomputer weltweit nach Betriebssystem im November 2024



Quelle:
TOP500
© Statista 2025

Weitere Informationen:
Weltweit

statista

Quelle: Statista,
Zugriff 24.02.2025

Prozess-Synchronisation

Was erwartet Sie in diesem Kapitel?

Computerkomponenten

- von-Neumann-Architektur
- Assembler

Betriebssysteme

- Prozesse / Tasks
- Multitasking
- Multithreading
- Scheduling-Verfahren

Prozess-Synchronisation

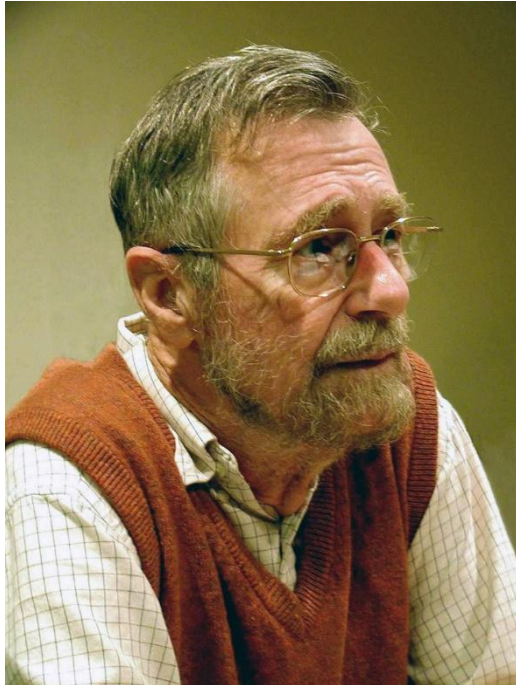
- Semaphore
- Monitor
- Deadlock



Edsger Wybe Dijkstra
(1930-2002)

Bildquelle: <https://beruhmte-zitate.de/autoren/edsger-wybe-dijkstra/>

Semaphore



Cooperating sequential processes
E.W. Dijkstra Archive.
Center for American History,
University of Texas at Austin. 1965

Table of Contents.	
Preface.	
0. Introduction	0
1. On the Nature of Sequential Processes.	1
2. Loosely Connected Processes.	10
2.1. A Simple Example.	11
2.2. The Generalized Mutual Exclusion Problem.	19
2.3. A Linguistic Interlude.	22
3. The Mutual Exclusion Problem Revisited.	26
3.1. The Need for a More Realistic Solution.	26
3.2. The Synchronizing Primitives.	28
3.3. The Synchronizing Primitives Applied to the Mutual Exclusion Problem.	30
4. The General Semaphore.	31
4.1. Typical Uses of the General Semaphore.	31
4.2. The Superfluity of the General Semaphore.	35
4.3. The Bounded Buffer.	39
5. Cooperation via Status Variables.	40
5.1. An Example of a Priority Rule.	41
5.2. An Example of Conversations	48
5.2.1. Improvements of the Previous Program.	61
5.2.2. Proving the Correctness.	68
6. The Problem of the Deadly Embrace.	73
6.1. The Banker's Algorithm.	75
6.2. The Banker's Algorithm Applied	77
7. Concluding Remarks.	82

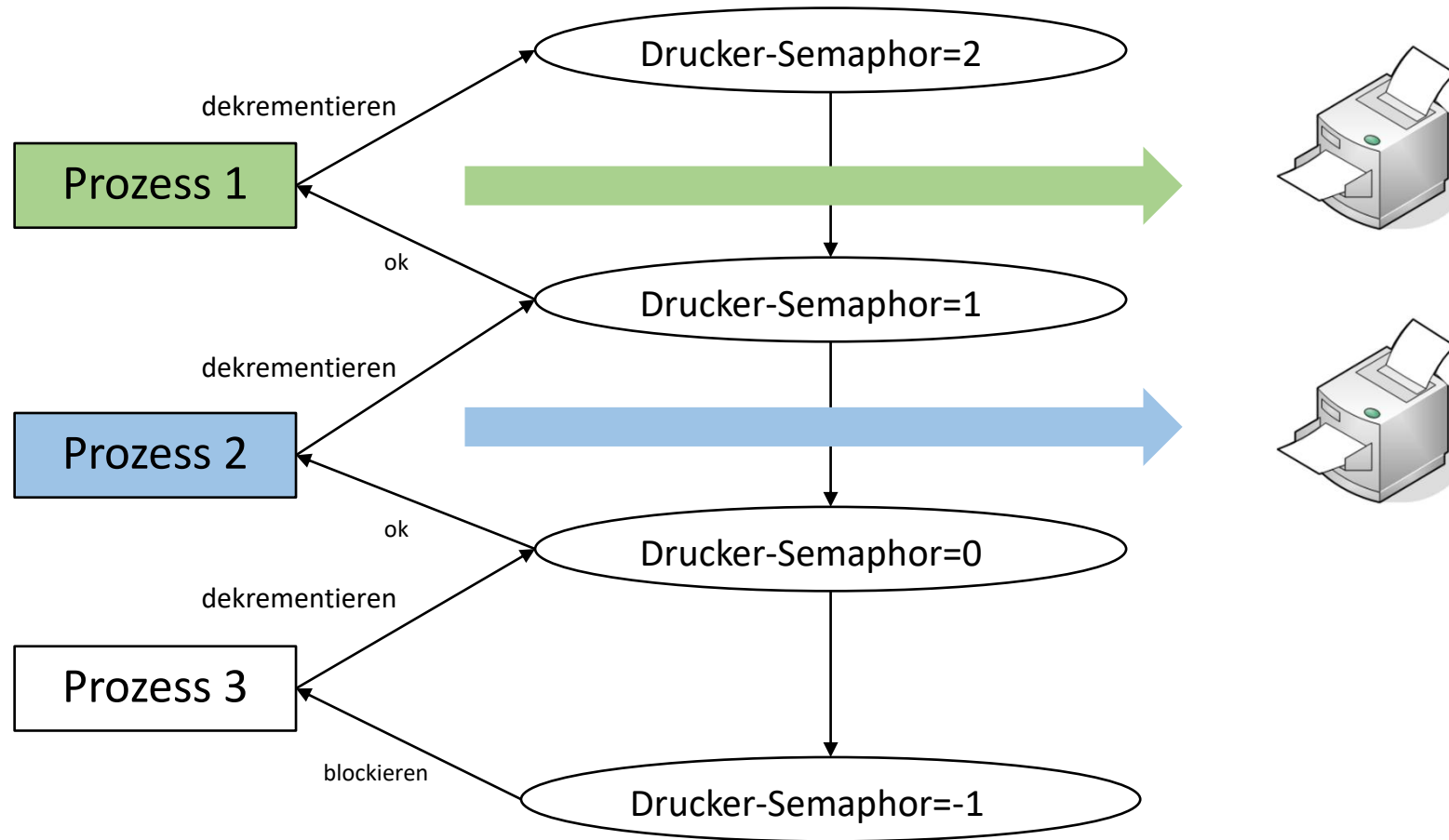
Bildquelle: Von Hamilton Richards - manuscripts of Edsger W. Dijkstra, University Texas at Austin, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=4204157>

Semaphore – Synchronisation von Prozessen

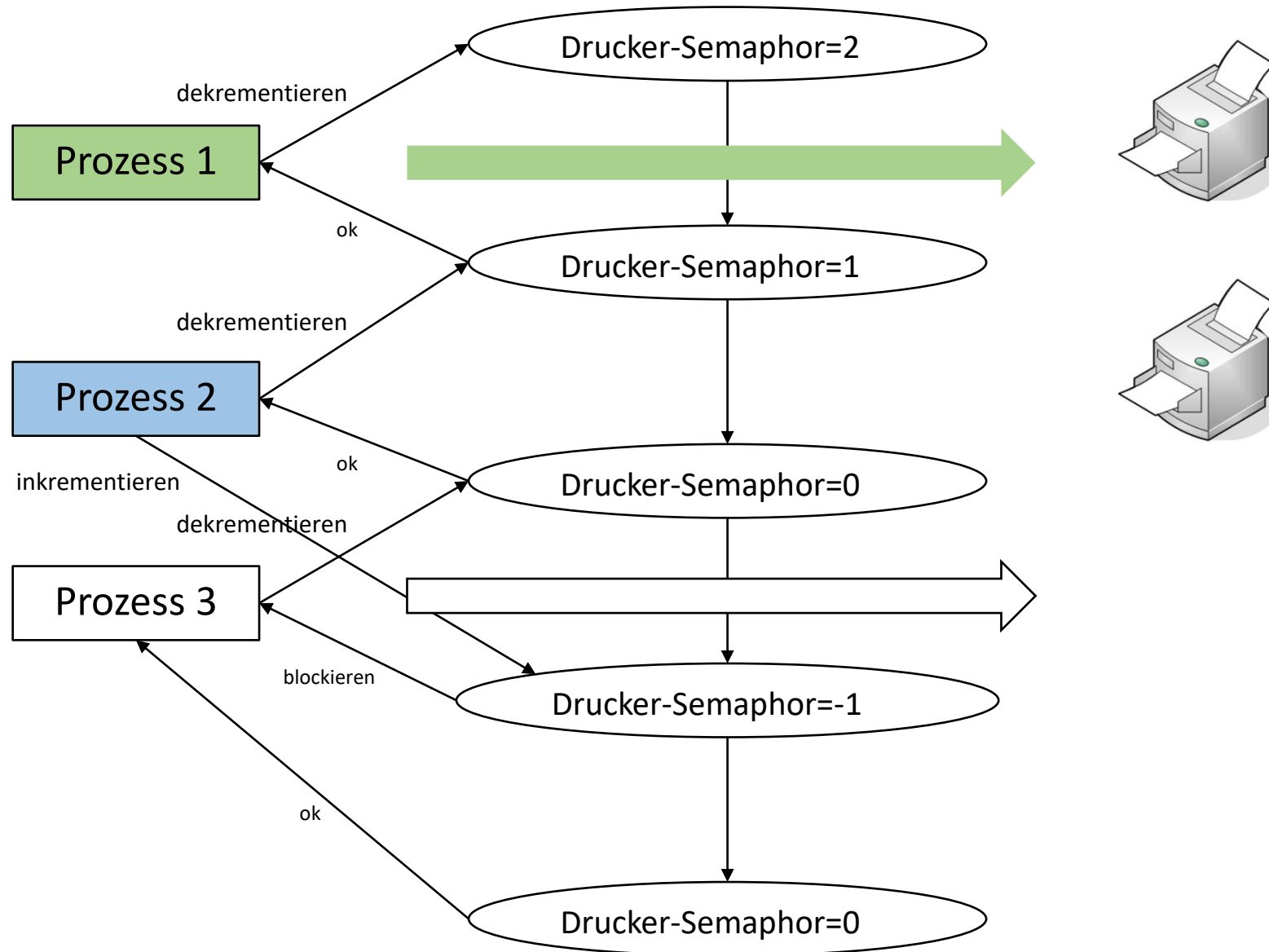
Ein Semaphor ist wie eine ganze Zahl, mit drei Unterschieden:

1. Wenn ein Semaphor erstellt wird, kann es mit einer beliebigen ganzen Zahl initialisiert werden. ABER danach sind die einzigen zulässigen Operationen Inkrement (Erhöhung um eins) und Dekrement (Verringerung um eins).
Man kann den aktuellen Wert des Semaphors nicht lesen.
2. Wenn ein Thread die Semaphore dekrementiert, blockiert sich der Thread bei negativem Ergebnis selbst und kann nicht fortfahren, bis ein anderer Thread die Semaphore erhöht.
3. Wenn ein Thread die Semaphore inkrementiert, wenn andere Threads warten, wird einer der wartenden Threads entsperrt.

Beispiel



Beispiel



Semaphore

- bestehen i.a. aus
 - einer Booleschen Variable oder
 - einer ganzzahligen Variable
 - und einer Warteschlange für die Tasks
- wartende Prozesse / Threads werden in eine Warteschlange eingetragen
- beim Zurückgeben des Semaphores wird eine eingetragene Task in der Warteschlange sofort wieder lafbereit

Sir Charles Antony Richard Hoare (*1934)

Operating
Systems

C. Weissman
Editor

Monitors: An Operating System Structuring Concept

C.A.R. Hoare
The Queen's University of Belfast

This paper develops Brinch-Hansen's concept of a monitor as a method of structuring an operating system. It introduces a form of synchronization, describes a possible method of implementation in terms of semaphores and gives a suitable proof rule. Illustrative examples include a single resource scheduler, a bounded buffer, an alarm clock, a buffer pool, a disk head optimizer, and a version of the problem of readers and writers.

Key Words and Phrases: monitors, operating systems, scheduling, mutual exclusion, synchronization, system implementation languages, structured multiprogramming
CR Categories: 4.31, 4.22

Copyright © 1974, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted, provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

This paper is based on an address delivered to IRIA, France, May 11, 1973. Author's address: Department of Computer Science, The Queen's University of Belfast, Belfast BT7 1NN, Northern Ireland.

549

1. Introduction

A primary aim of an operating system is to share a computer installation among many programs making unpredictable demands upon its resources. A primary task of its designer is therefore to construct resource allocation (or scheduling) algorithms for resources of various kinds (main store, drum store, magnetic tape handlers, consoles, etc.). In order to simplify his task, he should try to construct separate schedulers for each class of resource. Each scheduler will consist of a certain amount of local administrative data, together with some procedures and functions which are called by programs wishing to acquire and release resources. Such a collection of associated data and procedures is known as a *monitor*; and a suitable notation can be based on the *class* notation of SIMULA67 [6].

```
monitorname: monitor
begin ... declarations of data local to the monitor;
  procedure procname (... formal parameters ...);
  begin ... procedure body ... end;
  ... declarations of other procedures local to the monitor;
  ... initialization of local data of the monitor ...
end;
```

Note that the procedure bodies may have local data, in the normal way.

In order to call a procedure of a monitor, it is necessary to give the name of the monitor as well as the name of the desired procedure, separating them by a dot:

```
monitorname.procname(... actual parameters ...);
```

In an operating system it is sometimes desirable to declare several monitors with identical structure and behavior, for example to schedule two similar resources. In such cases, the declaration shown above will be preceded by the word **class**, and the separate monitors will be declared to belong to this class:

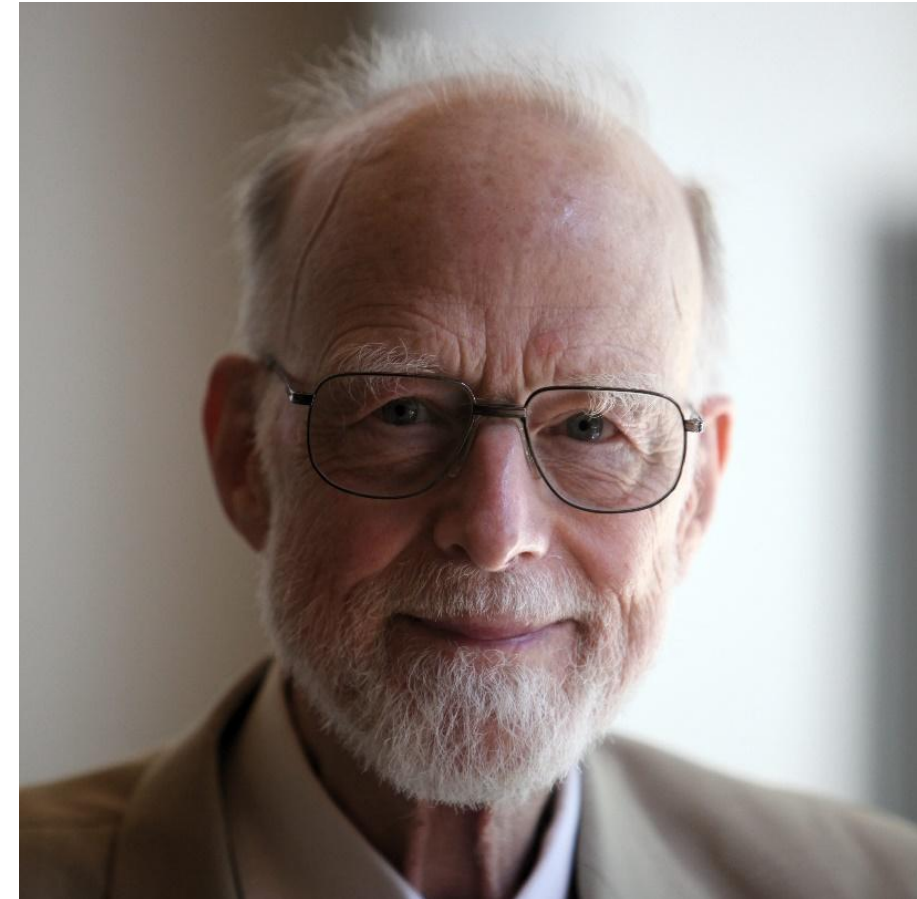
```
monitor 1, monitor 2: classname;
```

Thus the structure of a class of monitors is identical to that described for a data representation in [13], except for addition of the basic word *monitor*. Brinch-Hansen uses the word *shared* for the same purpose [3].

The procedures of a monitor are common to all running programs, in the sense that any program may at any time attempt to call such a procedure. However, it is essential that only one program at a time actually succeed in entering a monitor procedure, and any subsequent call must be held up until the previous call has been completed. Otherwise, if two procedure bodies were in simultaneous execution, the effects on the local variables of the monitor could be chaotic. The proce-

Communications
of
the ACM

October 1974
Volume 17
Number 10



Bildquelle: Photograph by Rama, Wikimedia Commons, Cc-by-sa-2.0-fr [CeCILL (http://www.cecill.info/licences/Licence_CeCILL_V2-en.html)

Per Brinch Hansen (1938 - 2007)

Das Monitor-Konzept wurde gleichzeitig von C.A.R. Hoare und Per Brinch Hansen 1973/74 entwickelt.



Bildquelle: http://www.wikiwand.com/en/Per_Brinch_Hansen

Monitor

Ein Monitor ist ein programmiersprachliches Konzept zur Synchronisation von Zugriffen zeitlich verschränkt oder parallel laufender Prozesse oder Threads auf gemeinsam genutzten Datenstrukturen oder Ressourcen.

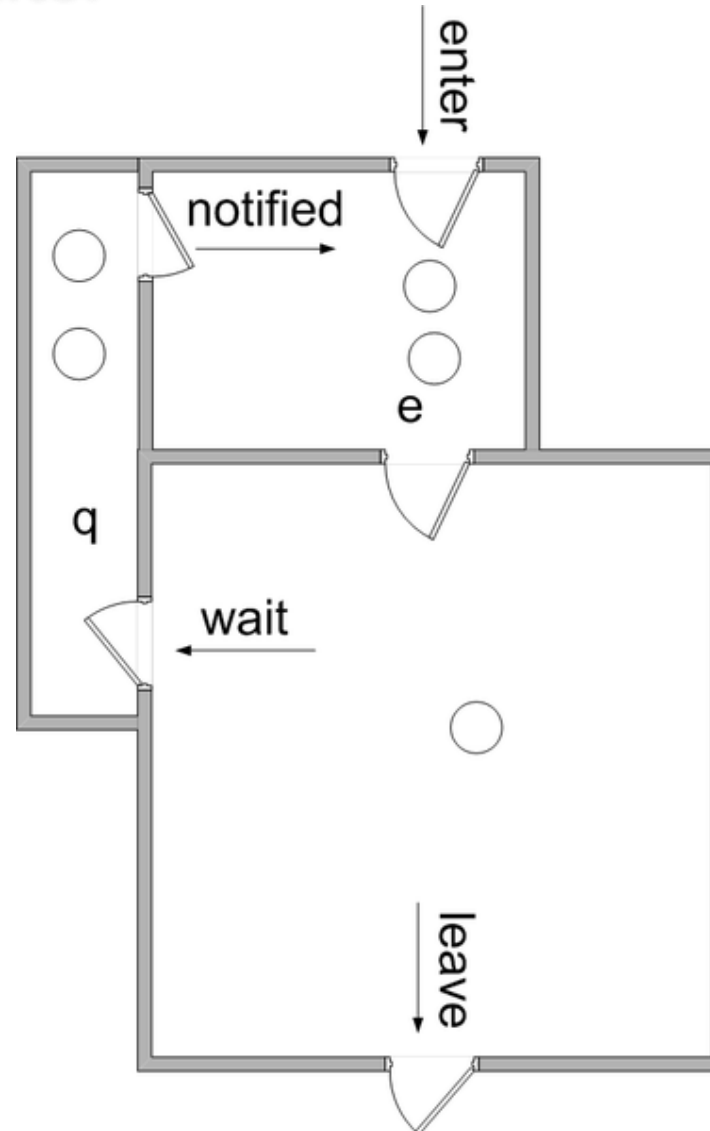
Inkonsistente Zustände der Datenstrukturen werden vermieden, ohne dass Programmierer Synchronisationsprimitive wie z. B. Semaphore explizit nutzen müssen.

Ein wechselseitiger Ausschluss bei Zugriffen auf die gemeinsam genutzte Datenstruktur wird erreicht, indem ein Compiler bei der Übersetzung eines Programmteils, das vom Programmierer mit Elementen der Programmiersprache als Monitor gekennzeichnet wurde, entsprechende Synchronisationsprimitive einfügt.

Das Konzept wird z. B. von den Programmiersprachen Ada, Modula, Concurrent Pascal oder Java realisiert.

Wikipedia

Monitor – Java style monitor



Bildquelle: User:Theodore.norvell [CC BY 3.0 (<https://creativecommons.org/licenses/by/3.0/>)], from Wikimedia Commons

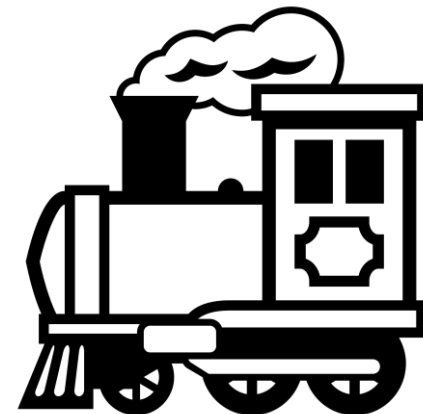
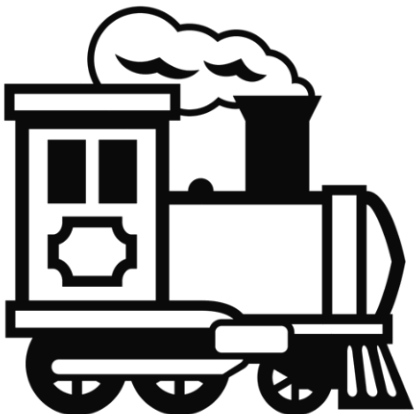
Beispiel für Java

```
class Something {  
    private SomeType sharedData;  
  
    public synchronized void function1 (...) {  
        ...  
    }  
    public synchronized void function2 (...) {  
        ...  
    }  
}
```

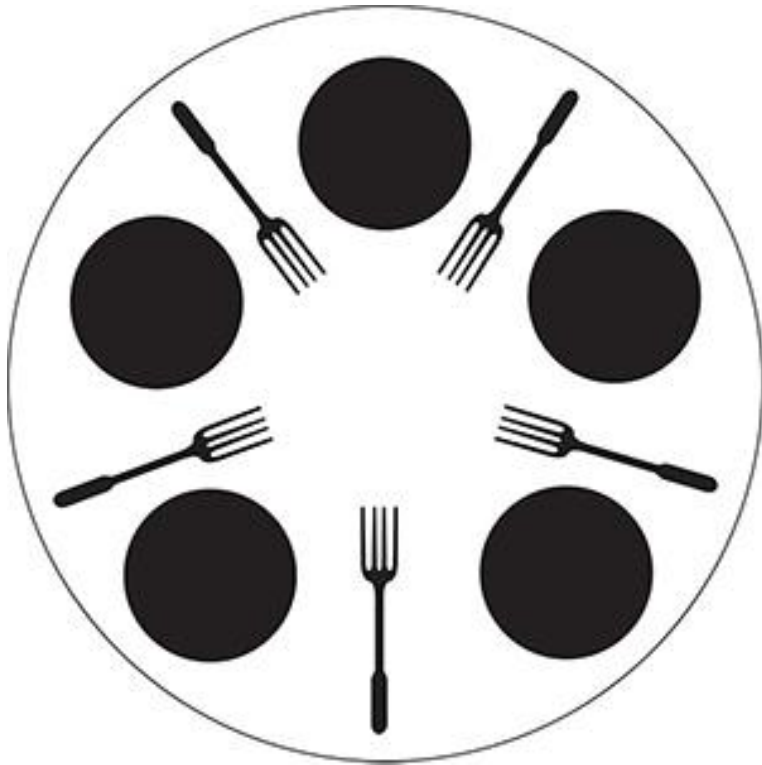
“When two trains approach each other
at a crossing, both shall come to a full
stop and neither shall start up again
until the other has gone.”

Gesetz in Kansas

A Treasury of Railroad Folklore, B.A. Botkin & A.F. Harlow, p. 381



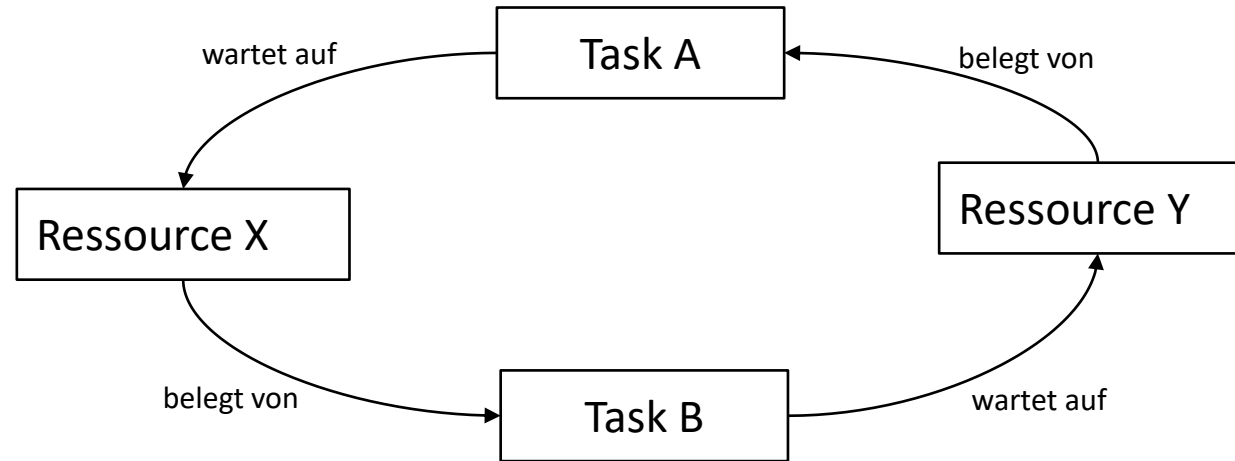
The Dining Philosophers Problem



Um einen runden Tisch sitzen 5 Philosophen. Jeder von ihnen besitzt einen Teller und eine Gabel zu seiner Linken.

Zum Essen benötigen die Philosophen jedoch 2 Gabeln, sie müssen also auch auf die Gabel ihres rechten Nachbarn zugreifen.

Circular-Wait-Situation



- Beide Tasks benötigen beide Ressourcen.
- Task A belegt Ressource Y und benötigt noch Ressource X.
- Task B belegt Ressource X und benötigt noch Ressource Y.
- Keiner der beiden Tasks kann arbeiten.
- Eine derartige Verklemmung wird **Deadlock** genannt.

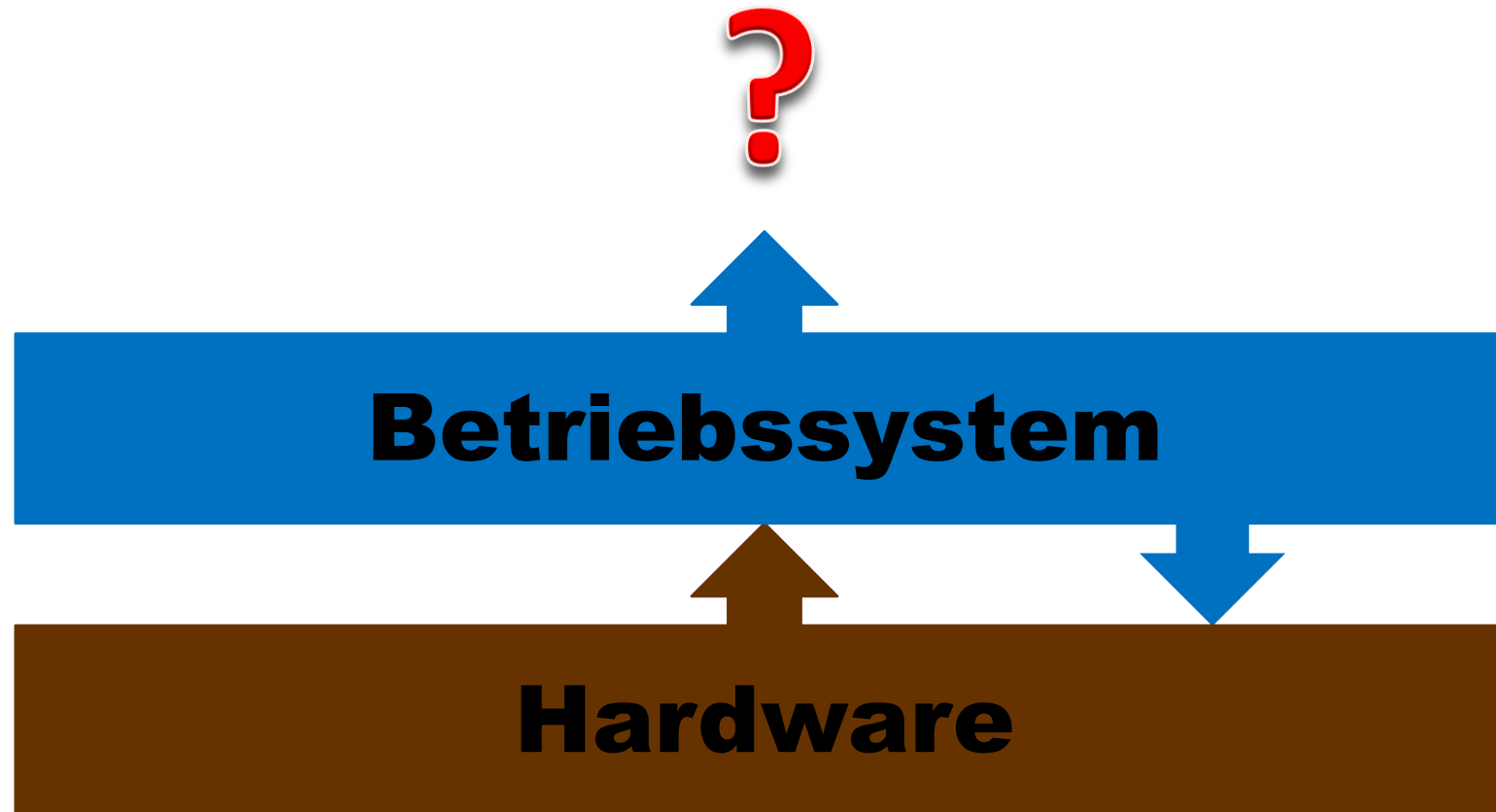
Erkennen und Auflösen von Deadlocks

Erkennen:

- Einsetzen von **Watch-Dogs** (Timer)
- ein Task muss einen Timer regelmässig neu stellen
- durch einen Deadlock wird das nicht mehr möglich
- der Timer läuft ab

Auflösen

- der betroffene Task wird abgebrochen und neu gestartet
- der Deadlock ist aufgelöst



Lernziele Computerkomponenten & Betriebssysteme

- ... wissen, was sich hinter den Begriffen CPU, Bus, von Neumannscher Flaschenhals verbirgt
- ... Computerkomponenten und ihre Aufgaben kennen
- ... Aufgaben von Betriebssystemen kennen
- ... Zustandsdiagramme verstehen
- ... Begrifflichkeiten Semaphore, Monitor, Deadlock kennen und Wirkungsweisen verstanden haben